

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ĐỒ ÁN THIẾT KẾ LUẬN LÝ

ĐỀ TÀI

“*THIẾT KẾ RISC CPU ĐƠN GIẢN*”

Instructor: Huỳnh Phúc Nghi (*CSE-HCMUT*)

Students: Nguyễn Văn Huynh - 2211315 (*Group L03*)
Nguyễn Duy Khiêm - 2211571 (*Group L03*)
Nguyễn Minh Hưng - 2211367 (*Group L03*)
Lê Quang Trung - 2213730 (*Group L03*)
Phan Đức Lợi - 2211946 (*Group L03*)

HO CHI MINH CITY, DECEMBER 2024



Contents

List of Figures	4
List of Tables	4
Member list & Workload	4
1 Giới thiệu	5
1.1 Tổng quan về đề tài	5
1.2 Công cụ và thiết bị sử dụng	5
1.3 Chức năng sản phẩm	5
2 Cơ sở lý thuyết	6
2.1 Kiến trúc CPU RISC	6
2.2 Các khối chức năng trong CPU	6
2.2.1 Program Counter	6
2.2.2 Register	6
2.2.3 Address Mux	6
2.2.4 Memory	6
2.2.5 ALU	7
2.2.6 Controller	7
2.3 Phương pháp thiết kế	8
3 Thiết kế hệ thống	9
3.1 Sơ đồ khối tổng thể	9
3.2 Mô tả chi tiết các khối chức năng	10
3.2.1 Program Counter	10
3.2.2 Register Instruction	10
3.2.3 Address Mux	11
3.2.3.a Chu kì câu lệnh đơn giản	11
3.2.3.b Flow chart	12
3.2.3.c Thiết kế sơ đồ khối	13
3.2.4 Memory	13
3.2.4.a Instruction memory	13
3.2.4.b Data memory	14
3.2.5 Accumulator	15
3.2.6 ALU	16
3.2.6.a Flow chart	16
3.2.6.b Sơ đồ khối	16
3.2.7 Controller	18
4 Hiện thực hệ thống	19
4.1 Program Counter	19
4.2 Address Mux	19
4.3 Instruction Memory	19
4.4 Instruction Register	19
4.5 Data Memory	19
4.6 Accumulator	19
4.7 ALU	20



4.8	Controller	20
5	Kiểm thử và mô phỏng	21
6	Đánh giá thiết kế và hướng phát triển trong tương lai	23
6.1	Đánh giá thiết kế	23
6.2	Khó khăn gặp phải	23
6.3	Hướng phát triển trong tương lai	23
7	Lời Kết	24
8	Tài liệu tham khảo	24



List of Figures

1	FMS STATE EACH INSTRUCTION	8
2	Sơ đồ khối của toàn bộ mạch	9
3	Sơ đồ Program Counter (PC)	10
4	Register Instruction	10
5	Cycle Instruction	11
6	Flowchart Address Mux	12
7	Sơ đồ khối Address Mux	13
8	Sơ đồ khối và flowchart của khối instruction memory	13
9	Sơ đồ khối và flowchart của khối data memory	14
10	sơ đồ khối của Accumulator Register	15
11	Flowchart ALU	16
12	Sơ đồ khối ALU	16
13	Sơ đồ khối và flowchart của khối controller.	18
14	Test Case	21
15	Testbench RTL	22

List of Tables

1	Member list & workload	4
2	Tính năng của RISC CPU Multi-cycle	5
3	Các lệnh CPU hỗ trợ	7



Member list & Workload

No.	Fullname	Student ID	Problems	% done
1	Nguyễn Văn Huynh	2211315	- Address Mux - ALU	125%
2	Nguyễn Minh Hưng	2211367	- Instruction Register - Program Counter - Accumulator Register	125%
3	Nguyễn Duy Khiêm	2211571	- Memory	125%
4	Lê Quang Trung	2213730	- Controller	125%
5	Phan Đức Lợi	2211946	- Không đóng góp	0%

Table 1: Member list & workload

1 Giới thiệu

1.1 Tổng quan về đề tài

Đề tài: Thiết kế RISC CPU đơn giản.

Trong đề tài này, nhóm sẽ thiết kế RISC CPU đơn giản với tập lệnh 8-bit gồm 3-bit opcode và 5-bit toán hạng. Tập lệnh gồm có 8 loại lệnh đơn giản và 32 không gian địa chỉ cho mỗi bộ nhớ instruction memory và data memory.

Bộ xử lý hoạt động dựa trên tín hiệu clock và reset. Chương trình sẽ dừng lại khi có tín hiệu HALT.

1.2 Công cụ và thiết bị sử dụng

Công cụ:

- **Phần mềm Vivado:** Được sử dụng để thiết kế, mô phỏng, và tổng hợp (synthesize) phần cứng của RISC CPU. Vivado cung cấp môi trường phát triển mạnh mẽ, hỗ trợ các ngôn ngữ mô tả phần cứng như Verilog hoặc VHDL.
- **GitHub:** Được sử dụng để quản lý mã nguồn (source code), lưu trữ phiên bản, và hỗ trợ làm việc nhóm hiệu quả. GitHub giúp theo dõi các thay đổi trong mã và dễ dàng phối hợp giữa các thành viên.

Thiết bị:

- **Kit Arty-Z7 20:** Đây là board phát triển FPGA dựa trên chip Xilinx Zynq-7000, được sử dụng để triển khai và kiểm tra thực tế thiết kế RISC CPU. Kit này hỗ trợ giao diện lập trình linh hoạt và có nhiều tính năng hữu ích để thực hiện các dự án liên quan đến xử lý tín hiệu số.

1.3 Chức năng sản phẩm

Các đặc điểm và chức năng chính của CPU được mô tả qua bảng bên dưới:

Tính năng	RISC CPU
Kiến trúc tập lệnh	Thiết kế tập lệnh 8-bit với 3-bit opcode và 5-bit toán hạng
Chu kỳ hoạt động	Multi-cycle (mỗi lệnh thực hiện trong nhiều chu kỳ xung)
Bộ nhớ	Bộ nhớ lệnh và bộ nhớ dữ liệu với không gian địa chỉ 32 ô
Xử lý tín hiệu điều khiển	Hỗ trợ tín hiệu Clock, Reset, và HALT
Độ chính xác xử lý dữ liệu	Hỗ trợ xử lý dữ liệu 8-bit và toán hạng 5-bit
Mô phỏng và triển khai	Mô phỏng trên Vivado và triển khai trên FPGA Arty-Z7 20
Khả năng mở rộng	Dễ dàng chỉnh sửa và mở rộng tập lệnh hoặc chức năng

Table 2: Tính năng của RISC CPU Multi-cycle

2 Cơ sở lý thuyết

2.1 Kiến trúc CPU RISC

2.2 Các khối chức năng trong CPU

2.2.1 Program Counter

- Counter là bộ đếm quan trọng dùng để đếm câu lệnh của chương trình. Ngoài ra còn có thể dùng để đếm các trạng thái của chương trình.
- Counter phải hoạt động khi có xung lên của *clk*.
- Reset kích hoạt mức cao, bộ đếm trở về 0.
- Counter với độ rộng số đếm là 5.
- Counter có chức năng load một số bất kì vào bộ đếm. Nếu không, bộ đếm sẽ hoạt động bình thường.

2.2.2 Register

- Tín hiệu đầu vào có độ rộng 8 bits.
- Tín hiệu *rst* đồng bộ và kích hoạt mức cao.
- Register phải hoạt động khi có xung lên của *clk*.
- Khi có tín hiệu load, giá trị đầu vào sẽ chuyển đến đầu ra.
- Ngược lại giá trị đầu ra sẽ không đổi.

2.2.3 Address Mux

- Khối Address Mux với chức năng của Mux sẽ chọn giữa địa chỉ lệnh trong giai đoạn nạp lệnh và địa chỉ toán hạng trong giai đoạn thực thi lệnh.
- Mux sẽ có độ rộng mặc định là 5.
- Độ rộng cần sử dụng parameter để vẫn thay đổi được nếu cần.

2.2.4 Memory

- Memory sẽ lưu trữ instruction và data.
- Memory sẽ lưu trữ instruction và data.
- Memory cần được thiết kế tách riêng chức năng đọc/ghi bằng cách sử dụng
- Single bidirectional data port. Không được đọc và ghi cùng lúc.
- 5-bit địa chỉ và 8-bit data.
- 1-bit tín hiệu cho phép đọc/ghi
- Memory phải hoạt động khi có xung lên của *clk*.

2.2.5 ALU

- ALU thực thi những phép toán số học. Phép tính được thực thi sẽ phụ thuộc vào toán tử của câu lệnh.
- ALU thực thi 8 phép toán trên số hạng 8-bit (inA và inB). Kết quả sẽ cho ra 8-bit output và 1-bit is_zero .
- is_zero bất đồng bộ nhằm cho biết input inA có bằng 0 hay không.
- Đầu vào opcode 3-bit sẽ quyết định phép toán nào được sử dụng như mô tả trong bảng sau:

Opcode	Mã	Hoạt động
HLT	000	Dừng hoạt động chương trình
SKZ	001	Trước tiên sẽ kiểm tra kết quả của ALU có bằng 0 hay không, nếu bằng 0 thì sẽ bỏ qua câu lệnh tiếp theo, ngược lại sẽ tiếp tục thực thi như bình thường
ADD	010	Cộng giá trị trong Accumulator vào giá trị bộ nhớ địa chỉ trong câu lệnh và kết quả được trả về Accumulator.
AND	011	Thực hiện AND giá trị trong Accumulator và giá trị bộ nhớ địa chỉ trong câu lệnh và kết quả được trả về Accumulator.
XOR	100	Thực hiện XOR giá trị trong Accumulator và giá trị bộ nhớ địa chỉ trong câu lệnh và kết quả được trả về Accumulator.
LDA	101	Thực hiện đọc giá trị từ địa chỉ trong câu lệnh và đưa vào Accumulator.
STO	110	Thực hiện ghi dữ liệu của Accumulator vào địa chỉ trong câu lệnh.
JMP	111	Lệnh nhảy không điều kiện, nhảy đến địa chỉ đích trong câu lệnh và tiếp tục thực hiện chương trình.

Table 3: Các lệnh CPU hỗ trợ

2.2.6 Controller

- Controller quản lý những tín hiệu điều khiển của CPU. Bao gồm nạp và thực thi lệnh.
- Controller phải hoạt động khi có xung lên của clk .
- Tín hiệu rst đồng bộ và kích hoạt mức cao.
- Tín hiệu opcode 3-bit tương ứng với ALU
- Controller cần có từ 8 trạng thái hoạt động trở lên

2.3 Phương pháp thiết kế

- Nhóm thiết kế máy trạng thái cho từng lệnh trong Controller

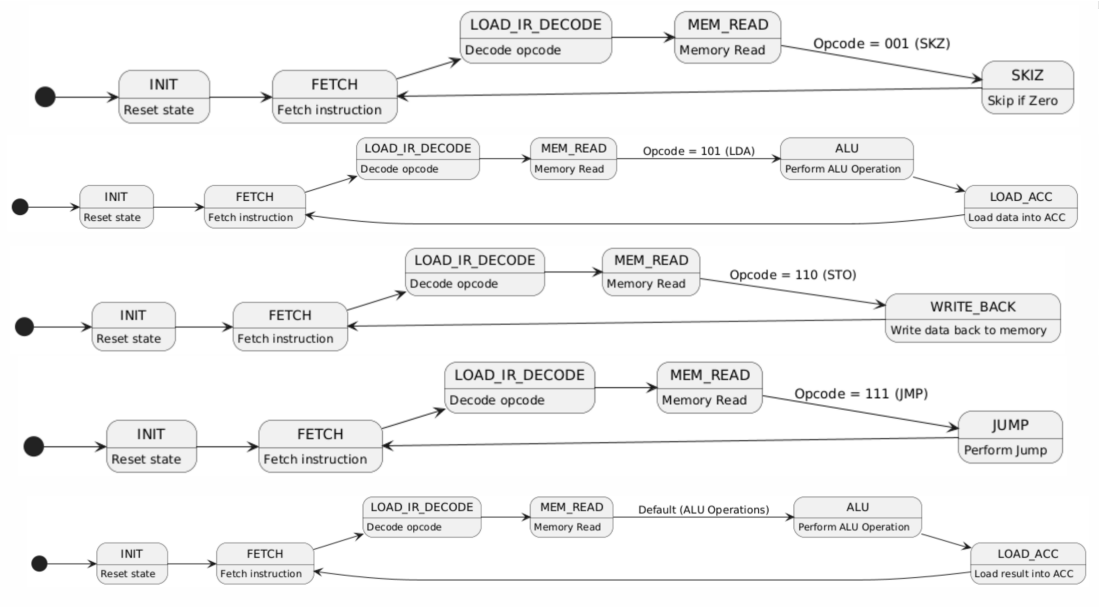


Figure 1: FMS STATE EACH INSTRUCTION

3 Thiết kế hệ thống

3.1 Sơ đồ khối tổng thể

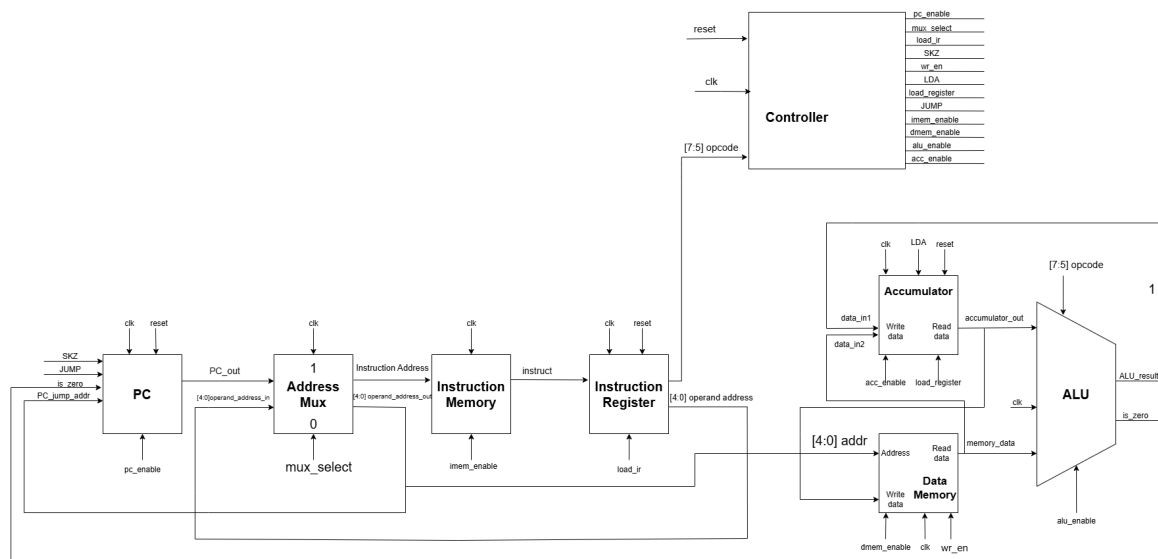


Figure 2: Sơ đồ khối của toàn bộ mạch

3.2 Mô tả chi tiết các khối chức năng

3.2.1 Program Counter

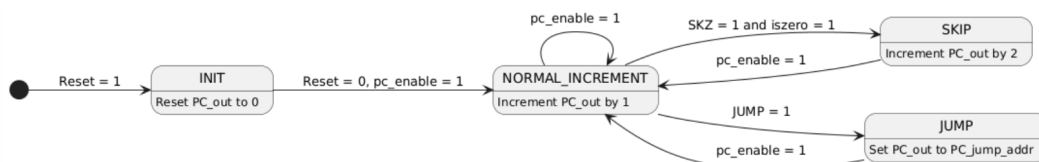


Figure 3: Sơ đồ Program Counter (PC)

- **NORMAL INCREMENT:** Thanh ghi PC tự động tăng khi kết thúc mỗi câu lệnh.
- **SKIP AND JUMP:** Khi có các lệnh đặc biệt như SKIZ và JUMP, thanh ghi PC sẽ chờ tín hiệu và tính toán địa chỉ trước khi thực hiện nhảy.

3.2.2 Register Instruction

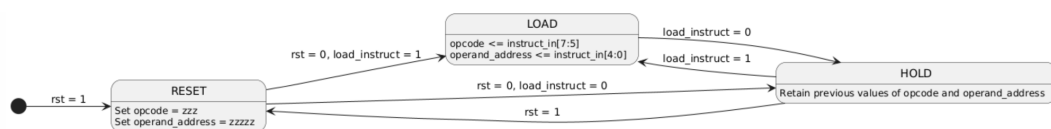


Figure 4: Register Instruction

- Register Instruction có nhiệm vụ cung cấp Opcode và instruction từ Instruction Memory

3.2.3 Address Mux

3.2.3.a Chu kì câu lệnh đơn giản

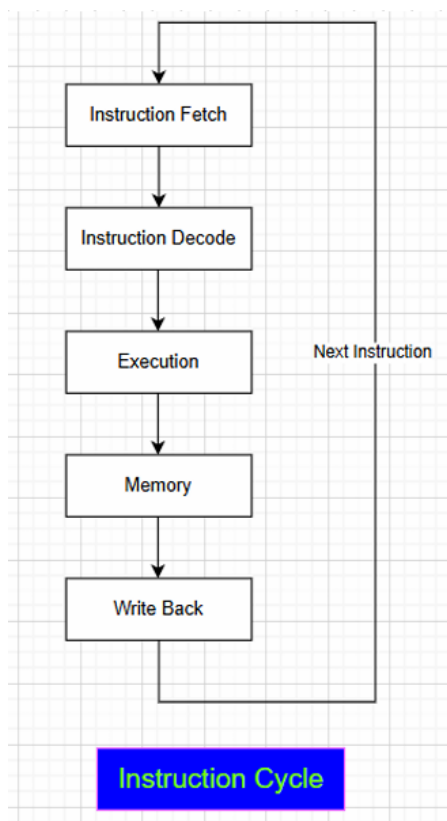


Figure 5: Cycle Instruction

- **Instruction Fetch (IF):** Nạp lệnh từ bộ nhớ tại địa chỉ được chỉ định bởi Program Counter (PC). Lệnh này sau đó sẽ được đưa vào thanh ghi lệnh (Instruction Register).
- **Instruction Decode (ID):** Giải mã lệnh đã nạp để xác định loại lệnh và các toán hạng liên quan. CPU sẽ phân tích lệnh này để xác định xem lệnh yêu cầu thao tác gì, ví dụ như toán học, logic, hay truy cập bộ nhớ.
- **Execution (EX):** Thực thi lệnh đã được giải mã, thường là thực hiện phép toán trong ALU (Arithmetic Logic Unit) hoặc kiểm tra các điều kiện.
- **Memory Access (MEM):** Đối với các lệnh liên quan đến bộ nhớ, CPU sẽ truy cập bộ nhớ để đọc dữ liệu (nếu lệnh là đọc) hoặc ghi dữ liệu (nếu lệnh là ghi).
- **WriteBack (WB):** Ghi kết quả của lệnh trở lại bộ nhớ hoặc các thanh ghi (như Accumulator) để hoàn tất chu kỳ lệnh.

3.2.3.b Flow chart

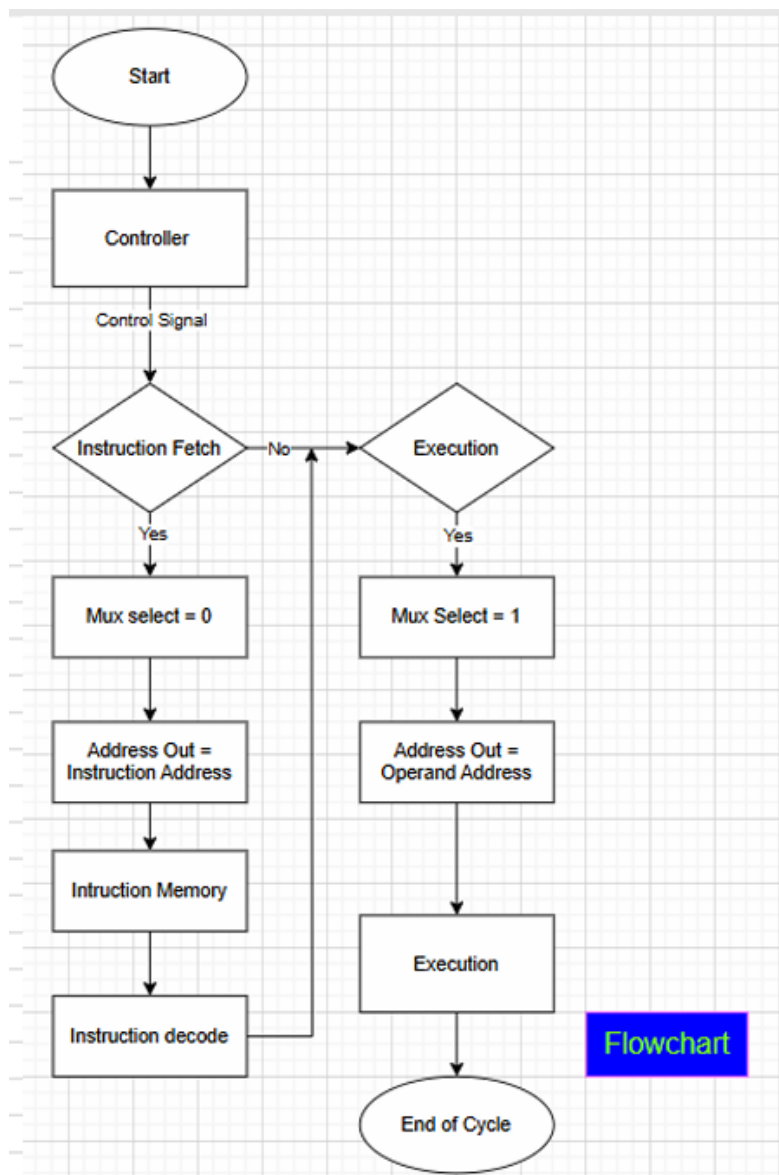


Figure 6: Flowchart Address Mux

Khối Controller sẽ sinh ra tín hiệu điều khiển (Mux select) để cho biết ta đang ở giai đoạn nào trong chu kỳ câu lệnh.

Nếu đang ở trạng thái Fetch, tín hiệu Mux select bằng 0, đầu ra của khối sẽ là địa chỉ câu lệnh, thông qua địa chỉ này ta sẽ vào Instruction Memory để lấy lệnh, sau đó Decode và thực thi câu lệnh.

Nếu đang ở giai đoạn Execution, tín hiệu Mux select bằng 1, đầu ra của khối sẽ là địa chỉ toán hạng, thông qua địa chỉ này ta sẽ vào Data Memory để lấy dữ liệu, sau đó thực hiện tính toán.

3.2.3.c Thiết kế sơ đồ khối

Dựa vào Flowchart trên, ta có thể xây dựng nên sơ đồ khối để đáp ứng yêu cầu chức năng của Address Mux như sau :

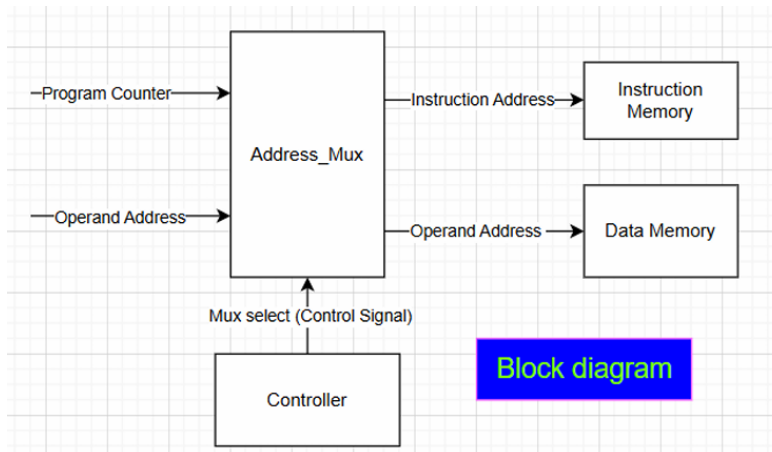


Figure 7: Sơ đồ khối Address Mux

3.2.4 Memory

3.2.4.a Instruction memory

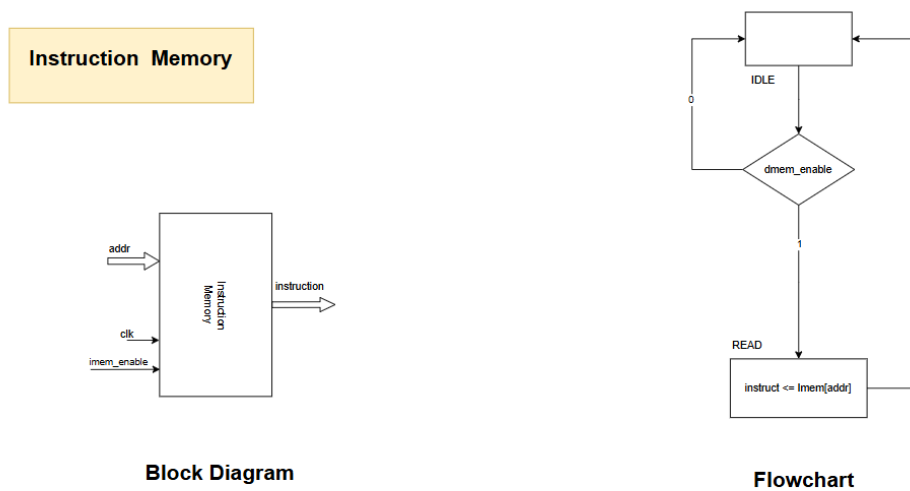


Figure 8: Sơ đồ khối và flowchart của khối instruction memory

Khối instruction memory có đặc tính read-only chỉ cho phép đọc và lấy lệnh ra từ bộ nhớ. So với đặc tả ban đầu, sẽ có thêm tín hiệu **imem_enable** vì khối memory được chia thành hai phần

là instruction và data. *imem_enable* là tín hiệu cho phép truy cập vào khối instruction memory, *data_enable* là tín hiệu cho phép truy cập vào khối data memory.

Đầu vào:

- Tín hiệu đồng bộ *clk*
- Tín hiệu cho phép truy cập vào khối *imem_enable*
- Địa chỉ lệnh *addr*

Đầu ra:

- Mã lệnh *instruction* 8-bit

3.2.4.b Data memory

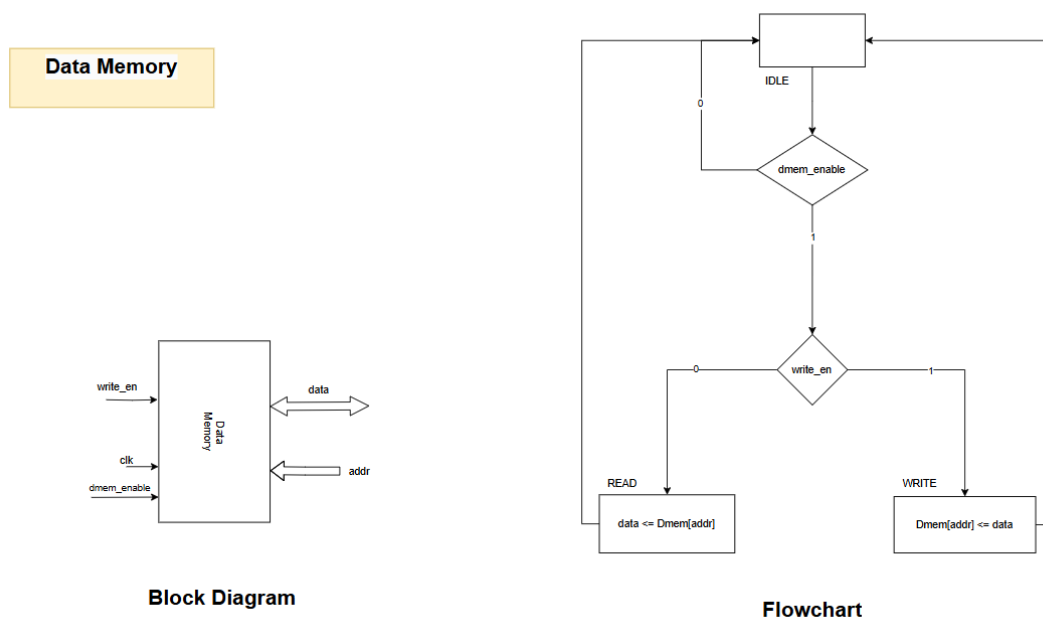


Figure 9: Sơ đồ khối và flowchart của khối data memory

Tương tự như instruction memory, *data_enable* là tín hiệu cho phép truy cập vào khối data memory. Khối data memory cho phép truy cập ở cả 2 chế độ read và write do đó cần tín hiệu *write_enable* để phân biệt 2 chế độ. Tín hiệu *write_enable* được set sẽ cho phép ghi dữ liệu từ đầu vào *data* vào bộ nhớ (trạng thái write), ngược lại sẽ cho phép đọc và lấy dữ liệu từ bộ nhớ ra đầu ra *data* (trạng thái read).

Đầu vào:

- Tín hiệu đồng bộ *clk*
- Tín hiệu cho phép truy cập vào khối *dmem_enable*

- Tín hiệu *write_enable* để phân biệt trạng thái read/write
- Địa chỉ dữ liệu *addr*
- Dữ liệu 8-bit cần ghi vào bộ nhớ *data* (nếu ở trạng thái write)

Đầu ra:

- Dữ liệu 8-bit đọc từ bộ nhớ *data* (nếu ở trạng thái read)

3.2.5 Accumulator

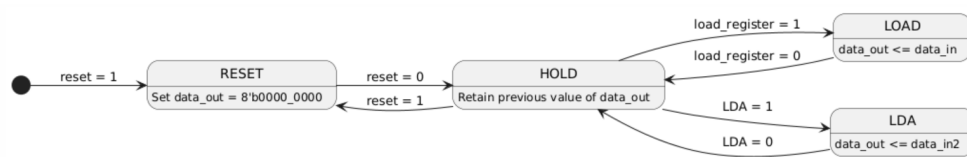


Figure 10: sơ đồ khối của Accumulator Register

- **LDA**: truyền dữ liệu từ khối Memory vào Accumulator
- **LOAD**: truyền dữ liệu sau khi tính toán ALU vào Accumulator

3.2.6 ALU

3.2.6.a Flow chart

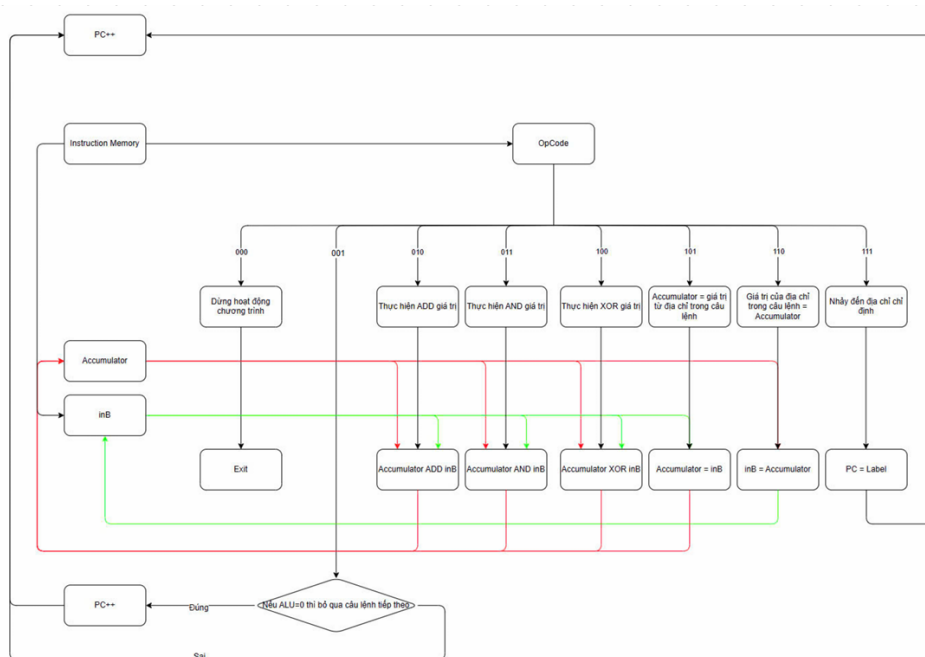


Figure 11: Flowchart ALU

3.2.6.b Sơ đồ khối

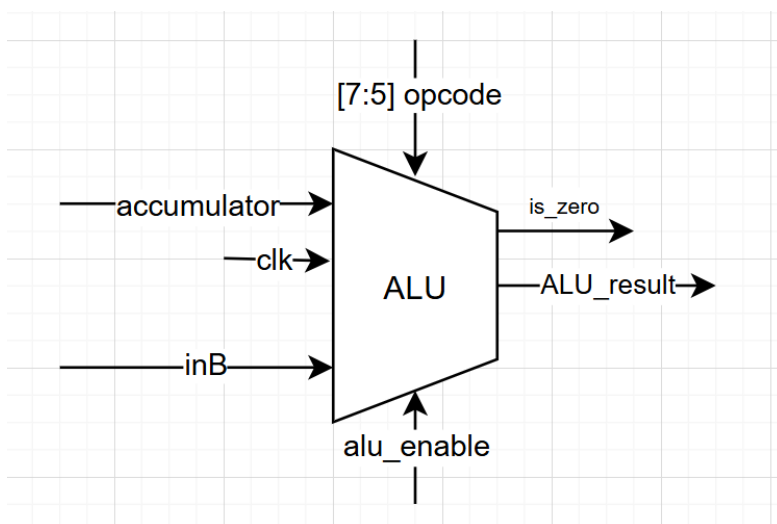


Figure 12: Sơ đồ khối ALU

Dựa vào Flowchart và sơ đồ khối chúng em đã đặc tả chức năng khối ALU theo đúng yêu cầu :

- Thực hiện các phép toán số học và logic.
- Các phép toán phụ thuộc vào opcode 3-bit của lệnh, bao gồm:
 - **HLT (000)**: Dừng hoạt động chương trình.
 - **SKZ (001)**: Kiểm tra kết quả ALU có bằng 0 không. Nếu bằng 0, bỏ qua lệnh tiếp theo; nếu không, tiếp tục thực hiện.
 - **ADD (010)**: Cộng giá trị trong *Accumulator* với giá trị từ bộ nhớ (*Memory*) và lưu kết quả vào *Accumulator*.
 - **AND (011)**: Thực hiện phép toán AND giữa giá trị *Accumulator* và giá trị từ *Memory*, lưu kết quả vào *Accumulator*.
 - **XOR (100)**: Thực hiện XOR giữa *Accumulator* và *Memory*, lưu kết quả vào *Accumulator*.
 - **LDA (101)**: Đọc giá trị từ *Memory* vào *Accumulator*.
 - **STO (110)**: Ghi giá trị từ *Accumulator* vào *Memory*.
 - **JMP (111)**: Lệnh nhảy không điều kiện đến địa chỉ mục tiêu trong lệnh.

Đầu vào

- Hai toán hạng 8-bit (*accumulator* và *inB*).
- Mã opcode 3-bit để xác định loại phép toán.
- tín hiệu **alu enable** được sinh ra từ khối Controller để kích hoạt ALU hoạt động.
- Controller phải hoạt động khi có xung lên của **clk**.

Đầu ra

- Kết quả 8-bit.
- Cờ **is_zero** (1-bit) cho biết kết quả ALU có bằng 0 hay không.

3.2.7 Controller

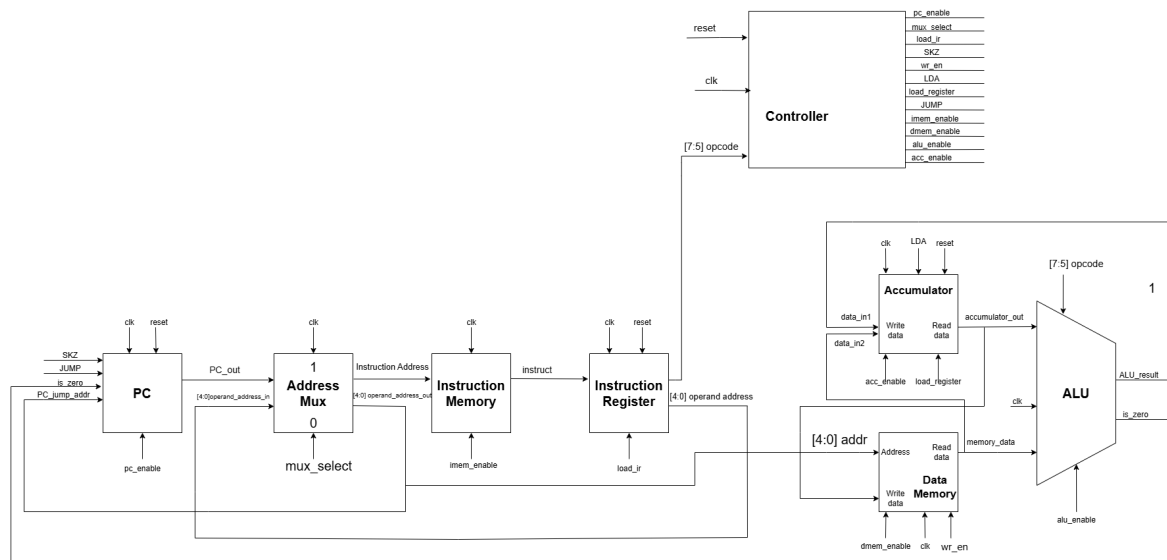


Figure 13: Sơ đồ khối và flowchart của khối controller.

Khối controller có nhiệm vụ chính là quản lý và điều phối hoạt động của các thành phần trong hệ thống dựa trên các tín hiệu điều khiển và opcode của lệnh hiện tại.

Đầu vào

- Controller sẽ hoạt động khi có xung lên của clk.
- Tín hiệu reset đồng bộ và kích hoạt mức cao.
- Tín hiệu opcode 3-bit tương ứng với ALU.

Đầu ra

- Tín hiệu pc_enable dùng để kích hoạt khối PC.
- Tín hiệu mux_select dùng để điều khiển khối Address Mux để chọn địa chỉ lệnh hoặc địa chỉ toán hạng.
- Tín hiệu imem_enable dùng để kích hoạt khối Instruction Memory để nạp lệnh.
- Tín hiệu load_ir dùng để kích hoạt khối Instruction Register cho phép giải mã lệnh.
- Các tín hiệu SKZ và JUMP sẽ kết hợp cùng với pc_enable để tăng địa chỉ PC với từng lệnh tương ứng.
- Tín hiệu dmem_enable dùng để kích hoạt khối Data Memory.
- Tín hiệu wr_en cho phép khối Data Memory được đọc hoặc ghi.
- Tín hiệu acc_enable dùng để kích hoạt khối Accumulator Register.

- Tín hiệu LDA kết hợp với acc_enable cho phép lệnh LDA ghi vào khối Accumulator Register.
- Tín hiệu load_register kết hợp với acc_enable cho phép ghi vào khối Accumulator Register (ngoại trừ lệnh LDA).
- Tín hiệu alu_enable dùng để kích hoạt khối ALU để tính toán.

4 Hiện thực hệ thống

4.1 Program Counter

- PC được thiết kế để cung cấp địa chỉ lệnh cho Instruction Memory trong mỗi chu kỳ lệnh.
- Chức năng chính: tăng địa chỉ PC sau mỗi chu kỳ clock.

4.2 Address Mux

- Chức năng chính: dùng để chọn địa chỉ lệnh trong giai đoạn nạp lệnh và địa chỉ toán hạng trong giai đoạn thực thi lệnh.

4.3 Instruction Memory

- Chức năng chính: lấy ra lệnh hiện tại từ địa chỉ do PC cung cấp để có thể dùng.

4.4 Instruction Register

- Chức năng chính: lấy lệnh được cung cấp từ Instruction Memory để giải mã bao gồm địa chỉ toán hạng và opcode của lệnh.

4.5 Data Memory

- Chức năng chính:
 - Đối với các lệnh số học, logic lấy địa chỉ toán hạng được cung cấp để lấy ra dữ liệu cung cấp cho ALU thực hiện tính toán.
 - Đối với lệnh LDA thì sẽ thực hiện đọc giá trị trong bộ nhớ để cung cấp cho Accumulator.
 - Đối với lệnh STO thì sẽ thực hiện ghi vào bộ nhớ.

4.6 Accumulator

- Accumulator là thanh ghi tạm thời để lưu trữ kết quả.
- Chức năng chính: Nhận kết quả từ ALU hoặc dữ liệu từ Data Memory.

4.7 ALU

- ALU thực hiện 8 phép toán dựa trên opcode.
- Chức năng chính:
 - Kết hợp hai dữ liệu (Input A và Input B) và trả kết quả.
 - Tín hiệu `is_zero` báo kết quả có bằng 0 hay không.

4.8 Controller

- Controller đảm bảo việc thực thi lệnh đúng thứ tự trong mỗi chu kỳ clock.
- Trình tự hoạt động:
 - Nạp lệnh: PC tăng, địa chỉ được chọn vào Instruction Memory.
 - Giải mã: Opcode chuyển tới controller.
 - Thực thi: ALU hoạt động và chuyển kết quả.
 - Ghi kết quả: Kết quả lưu trong Accumulator hoặc ghi vào Data Memory.

5 Kiểm thử và mô phỏng

Test case :

```

/*****
* Test program
*
* Kết quả cần có: Chương trình sau kết thúc (halt) ở lệnh địa chỉ 17(hex)
*****/

//opcode_operand // addr assembly code
//-----//-----
@00 111_11110 // 00 BEGIN: JMP TST_JMP
    000_00000 // 01 HLT
    000_00000 // 02 HLT
    101_11010 // 03 JMP_OK: LDA DATA_1
    001_00000 // 04 SKZ
    000_00000 // 05 HLT
    101_11011 // 06 LDA DATA_2
    001_00000 // 07 SKZ
    111_01010 // 08 JMP SKZ_OK
    000_00000 // 09 HLT
    110_11100 // 0A SKZ_OK: STO TEMP
    101_11010 // 0B LDA DATA_1
    110_11100 // 0C STO TEMP
    101_11100 // 0D LDA TEMP
    001_00000 // 0E SKZ
    000_00000 // 0F HLT
    100_11011 // 10 XOR DATA_2
    001_00000 // 11 SKZ
    111_10100 // 12 JMP XOR_OK
    000_00000 // 13 HLT
    100_11011 // 14 XOR_OK: XOR DATA_2
    001_00000 // 15 SKZ
    000_00000 // 16 HLT
    000_00000 // 17 END: HLT
    111_00000 // 18 JMP BEGIN

@1A 00000000 // 1A DATA_1: (giá trị hằng 0x00)
    11111111 // 1B DATA_2: (giá trị hằng 0xFF)
    10101010 // 1C TEMP: (biến khởi tạo với giá trị 0xAA)

@1E 111_00011 // 1E TST_JMP: JMP JMP_OK
    000_00000 // 1F HLT

```

Figure 14: Test Case

TestBench :

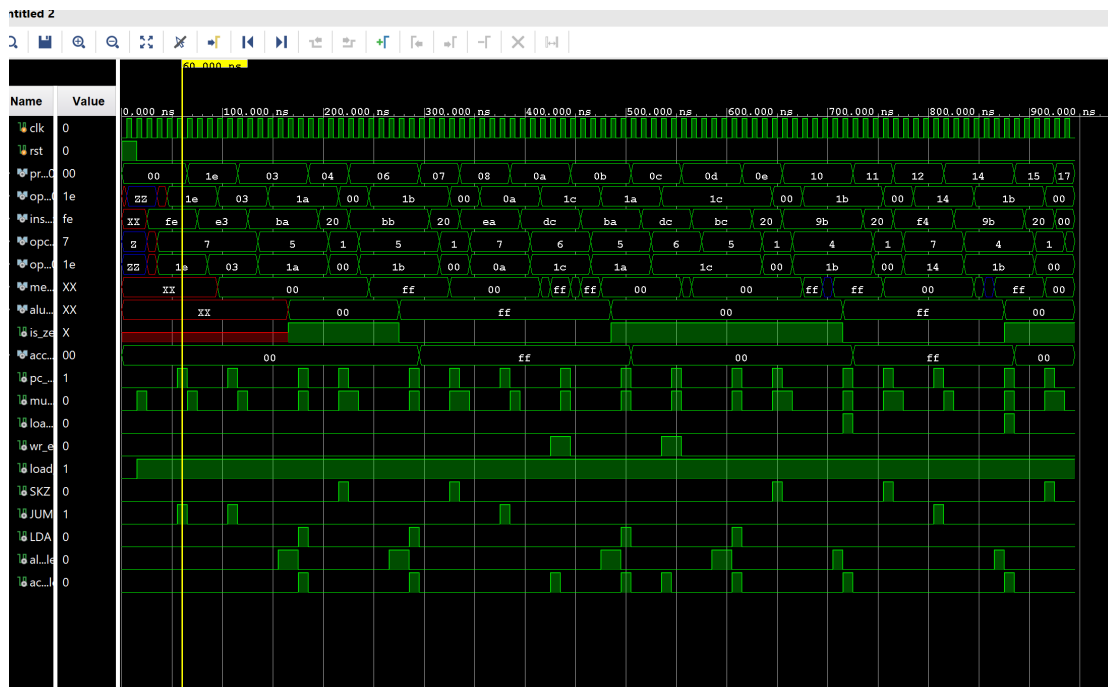


Figure 15: Testbench RTL

- Đầu tiên ta sẽ gặp lệnh JUMP (Opcode là 111) tại địa chỉ 0x00. Địa chỉ JUMP đến sẽ là 0x1E.
- Tại địa chỉ 0x1E ta tiếp tục gặp lệnh JUMP và nhảy đến địa chỉ 0x03.
- Tại địa chỉ 0x03 ta gặp lệnh LDA (Opcode là 101) load giá trị 0x00 vào Accumulator.
- Tiếp theo là đến địa chỉ 0x04 ta gặp lệnh SKZ(Opcode 001). Do kết quả ALU lúc này là 0x00 nên PC sẽ nhảy đến địa chỉ 0x06.
- Tại địa chỉ 0x06 ta gặp lệnh LDA (Opcode là 101) load giá trị 0xFF vào Accumulator.
- Tiếp theo tại địa chỉ 0x07 ta gặp lệnh SKZ (Opcode 001), do lúc này giá trị của ALU khác 0 nên ta sẽ tiếp tục đến địa chỉ tiếp theo 0x08.
- Tại địa chỉ 0x08 ta gặp lệnh JUMP (Opcode 111) nhảy đến địa chỉ 0x0A.
- Tại địa chỉ 0x0A ta gặp lệnh STO (Opcode 110) lưu giá trị của Accumulator (0xFF) vào địa chỉ 0x1C.
- Tại địa chỉ 0x0B ta gặp lệnh LDA (Opcode là 101) load giá trị 0x00 vào Accumulator.
- Tại địa chỉ 0x0C ta gặp lệnh STO (Opcode 110) lưu giá trị của Accumulator (0x00) vào địa chỉ 0x1C.
- Tại địa chỉ 0x0D ta gặp lệnh LDA (Opcode là 101) load giá trị 0x00 vào Accumulator.

- Tiếp theo là đến địa chỉ 0x0E ta gặp lệnh SKZ (Opcode 001). Do kết quả ALU lúc này là 0x00 nên PC sẽ nhảy đến địa chỉ 0x10.
- Tại địa chỉ 0x10 ta gặp lệnh XOR (Opcode là 100) xor giá trị 0x00 trong Accumulator và giá trị tại địa chỉ 0x1B (0xFF) ra kết quả 0xFF.
- Tiếp theo tại địa chỉ 0x11 ta gặp lệnh SKZ (Opcode 001), do lúc này giá trị của ALU khác 0 nên ta sẽ tiếp tục đến địa chỉ tiếp theo 0x12.
- Tại địa chỉ 0x12 ta gặp lệnh JUMP và nhảy đến địa chỉ 0x14.
- Tại địa chỉ 0x14 ta gặp lệnh XOR (Opcode là 100) xor giá trị 0xFF trong Accumulator và giá trị tại địa chỉ 0x1B (0xFF) ra kết quả 0x00.
- Tiếp theo là đến địa chỉ 0x15 ta gặp lệnh SKZ (Opcode 001). Do kết quả ALU lúc này là 0x00 nên PC sẽ nhảy đến địa chỉ 0x17.
- Tại địa chỉ 0x17 ta gặp lệnh HAL (Opcode 000) chương trình kết thúc tại địa chỉ này.

6 Đánh giá thiết kế và hướng phát triển trong tương lai

6.1 Đánh giá thiết kế

Đánh giá với bộ testcase đề xuất, kết quả của nhóm như sau:

- Pass được behaviour simulation
- Synthesis simulation không hoạt động như ý muốn (nguyên nhân xuất phát từ các timing warning, schematic ở synthesis khác nhiều với schematic ban đầu)
- Kết quả chạy trên mạch thật hoạt động không đúng. (kết quả giống với synthesis)

6.2 Khó khăn gặp phải

Trong quá trình thực hiện đồ án, nhóm gặp phải một vài khó khăn về chuyên môn và phi chuyên môn:

- Vấn đề liên quan đến synthesis và timing khiến mạch hoạt động không đúng.
- Kiến thức về kiến trúc CPU vẫn còn hạn hẹp, chưa tối ưu được tài nguyên.
- Sự thay đổi trong thành viên nhóm, kế hoạch bị trì trệ và dẫn đến kết quả không tốt.

6.3 Hướng phát triển trong tương lai

- Cải thiện kiến thức và kỹ năng:
 - Tăng cường nghiên cứu sâu hơn về kiến trúc CPU, tập trung vào các kỹ thuật tối ưu hóa tài nguyên.
 - Học thêm về các phương pháp xử lý timing và cách khắc phục các vấn đề liên quan đến synthesis simulation.
- Hoàn thiện và tối ưu hóa mạch:

- Điều chỉnh lại thiết kế mạch để xử lý tốt hơn các vấn đề liên quan đến timing.
- Tiến hành thêm các thử nghiệm thực tế trên nhiều loại phần cứng để đánh giá hiệu suất tổng thể.
- Mở rộng chức năng:
 - Thêm các lệnh và chức năng mới vào CPU để tăng tính ứng dụng, chẳng hạn như hỗ trợ các phép toán phức tạp hoặc giao tiếp ngoại vi.
 - Thiết kế bộ nhớ cache đơn giản để cải thiện tốc độ truy xuất dữ liệu.
- Ứng dụng thực tế: Triển khai mô hình CPU này vào các hệ thống nhúng đơn giản hoặc ứng dụng nhỏ để đánh giá hiệu suất thực tế.

7 Lời Kết

Nhìn chung nhóm đã hiện thực được yêu cầu của đề án ở mức RTL nhưng vẫn còn nhiều hạn chế khi tổng hợp logic từ RTL qua Synthesis dẫn đến delay mạch. Qua đây nhóm học được thêm nhiều cách có được một đề án hoàn chỉnh hơn bằng cách chạy phân tích nhiều phần để có được đánh giá chính xác về khả năng ứng dụng của mạch khi chạy thực tế và nhóm hiểu thêm được việc viết code theo rule quan trọng và ảnh hưởng rất lớn đến việc chạy trên mạch thật.

Link github được nhóm cập nhật [tại đây](#)

8 Tài liệu tham khảo

- <https://www.fpga4student.com/>
- <https://www.fpga4fun.com/>
- <https://sg.element14.com/>
- <https://www.hackster.io/>
- <https://www.instructables.com/circuits/howto/fpga/>